GenSpark

Coding best practices

Writing code that works is only part of the job. Writing code that other people can read, maintain, and safely change six months from now is a different skill — and it's the one that separates professional software engineering from just getting something to run. This article covers the core habits that matter most, especially in a role where you're often the one person on the ground representing the quality of the work.

Why Best Practices Exist

Code is read far more often than it's written. You might write a function once, but over its lifetime it will be read, debugged, and modified by other engineers — and often by you, months later, after you've forgotten the details. Best practices exist to reduce the cost of that ongoing reading and change, not to satisfy some abstract standard of "clean code."

In a client-facing role, there's an added dimension: your code often becomes something the client's own team inherits and maintains after you're gone. Sloppy code doesn't just create friction for you — it creates a lasting impression of the work you and your organization delivered.

Naming Things Clearly

This sounds trivial, but poor naming is one of the most common sources of confusion in real codebases. A variable called `x` or a function called `process()` tells the next reader nothing about what it does.

Compare:

```
def calc(a, b):
    return a * b * 0.08
```

to:

```
def calculate_sales_tax(price, quantity):
    tax_rate = 0.08
    return price * quantity * tax_rate
```

The second version tells you what it does without needing a comment. Good naming is a form of documentation that never goes out of date, because it lives inside the logic itself.

Keep Functions Focused

A function should generally do one thing. When a function grows to handle multiple responsibilities — validating input, doing a calculation, saving to a database, sending a notification — it becomes harder to test, harder to reuse, and harder to reason about when something breaks.

If you find yourself writing a function and thinking "and then it also needs to...", that's often a sign it should be split into smaller functions, each with a clear, single purpose.

Don't Repeat Yourself (DRY)

If you find yourself copying and pasting a block of code to use it in two places, that's usually a signal to extract it into a shared function instead. Duplicated code means duplicated bugs — if you fix an issue in one copy, you can easily forget to fix it in the other. Consolidating logic into one place means it only needs to be right once.

Comment the "Why," Not the "What"

New engineers often over-comment, explaining what the code does line by line. If the code is written clearly, the "what" should already be obvious from reading it. Comments are most valuable when they explain *why* a decision was made — especially when that reason isn't obvious from the code itself.

```
# Using a 500ms delay here because the third-party API
# rate-limits requests faster than that
time.sleep(0.5)
```

That comment earns its place. A comment like `# loop through the list` above a for-loop does not.

Consistent Formatting

Consistent indentation, spacing, and structure make code easier to scan. Most languages and teams adopt a style guide (PEP 8 for Python, for example) and use automated formatters to enforce it, so this isn't something you need to manually police — but you should understand why it matters: a codebase where every file looks different is harder for anyone to navigate quickly.

Handle Errors Deliberately

Code will encounter unexpected situations — a file that doesn't exist, a network call that times out, input that doesn't match what you expected. Best practice is to anticipate likely failure points and handle them deliberately, rather than letting the program crash with an unhelpful error, or silently ignoring the problem and letting bad data flow further into the system.

Write Code for the Next Person

A useful habit: before finishing a piece of work, ask yourself whether someone unfamiliar with what you just did could read it and understand both what it does and why it exists. If the answer is no, that's usually worth fixing before moving on — not after a client or teammate has to ask you.

Comprehension Check

1. Why is naming considered a form of documentation?
2. What's the risk of duplicating a block of code instead of extracting it into a shared function?
3. Give an example of a comment that explains "why" rather than "what," different from the one in this article.

← Previous: What is Full Stack Development

Next: Git workflows and PR hygiene →

0 comments so far

6

Add a comment...

Comment cannot be empty

Comment

No comments yet. Be the first to start the discussion!